FACULDADE DE ENGENHARIA DA UNIVERSIDADE DO PORTO

<Dissertation Title>

<Author's Full Name>

WORKING VERSION

Mestrado em Engenharia Eletrotécnica e de Computadores

Supervisor: Prof. <Name of the Supervisor>

July 30, 2025

Resumo

TO BE MADE AT THE END

Abstract

TO BE MADE AT THE END

UN Sustainable Development Goals

The United Nations Sustainable Development Goals (SDGs) provide a global framework to achieve a better and more sustainable future for all. It includes 17 goals to address the world's most pressing challenges, including poverty, inequality, climate change, environmental degradation, peace, and justice. n the context of this report, the implementation of Industry 4.0 technologies, particularly Human-Robot Collaboration (HRC) and Digital Twins, aligns with several SDGs, most notably:

- **Goal 9: Industry, Innovation, and Infrastructure** – The integration of advanced technologies such as HRC and Digital Twins into manufacturing processes supports innovation, fosters infrastructure development, and promotes resilient industrialization.
- **Goal 8: Decent Work and Economic Growth** – Human-robot collaboration can enhance productivity and create safer, more sustainable workplaces, contributing to decent work conditions and sustainable economic growth.
- **Goal 12: Responsible Consumption and Production** – Digital Twins and simulation models can optimize resource use, reduce waste, and improve the sustainability of industrial production processes, directly contributing to responsible consumption and production.
- **Goal 13: Climate Action** – By enabling energy-efficient processes and predictive maintenance through Digital Twins, Industry 4.0 technologies can help reduce the environmental footprint of manufacturing, supporting efforts in climate action.

Acknowledgements

I would like to express my deepest gratitude to my supervisor, Professor **António Almeida**, and to my co-supervisor, Professor **Américo Azevedo**. I am deeply grateful to my family and friends for their unwavering support. Finally, I would like to thank all the individuals and institutions that have supported me.

<João Pedro Caldas Ferreira>

"No one knows what the future holds. That's why its potential is infinite."

Okabe Rintarou

Contents

List of Figures

List of Tables

List of Acronyms

ABM	Agent-Based Modeling
COBOTS	Collaborative Robots
CPS	Cyber-Physical Systems
DT	Digital Twin
DPs	Design Principles
HRC	Human-Robot Collaboration
ISO	International Organization for Standardization
IoT	Internet of Things
MSL	Modelica Standard Library
DSS	Decision-Support Systems
DES	Discrete-Event Simulation
I/O	Inputs and Outputs
KPI	Key Performance Indicators
MTBF	Mean Time Between Failures
MTTR	Mean Time To Repair
MTBM	Mean Time Between Maintenance
TSLM	Time Since Last Maintenance
MTM	Mean Time for Maintenance
PCB	Printed Circuit Board

Chapter 1

Development of the Simulation Framework

This chapter describes the conceptual and technical development of the simulation framework designed to model and simulate assembly lines. The framework was designed to support scenario evaluation and decision-making within industrial assembly cells, while maintaining a high degree of versatility and customization. Its open-source foundation facilitates ongoing development and adaptation to meet contemporary and future manufacturing needs.

It presents the structure and implementation of the simulation tool, along with the underlying modeling principles, the reasoning behind the selected toolchain, and the design strategies adopted to ensure extensibility, realism, and usability. The simulation logic and graphical interface were developed from scratch to meet the specific requirements of the Bosch Ovar assembly environment, ensuring ease of use and facilitating upgrades and changes.

1.1 Goals and Design Principles

The primary objective of the proposed framework is to provide a flexible, configurable, and realistic simulation environment to support scenario analysis and decision-making in industrial assembly cells. The simulation framework serves as a decision-support tool, allowing engineers and planners to manually model the production systems before running the simulations. This modeling process is streamlined through a graphical interface specifically tailored to the concepts and structures used within Bosch assembly lines.

Several key design principles were adopted throughout the development of the framework:

- **Modularity** - The simulation system is composed of distinct modules, simulation logic, graphical interface, data logging, and statistical methods. This approach promotes maintainability and extensibility, facilitating future integrations, updates, and upgrades.

- **Interactive Visual Model Construction** - Users can craft the simulation models by interacting with a 2D graphical interface that allows dragging, connecting, and configuring elements.
- **Model Validation Mechanism** - To ensure simulation reliability, the framework includes a logic validation function that verifies essential conditions and model consistency before execution.
- **Open-Source Foundation** - The framework was built entirely using open-source technologies, granting full access to the simulation logic and enabling future adaptation to evolving requirements without licensing restrictions.
- **Configurability** - The tool enables users to modify parameters such as the number of operators in an assembly line, the integration of collaborative robots, processing times, movement durations, buffer sizes, and routing logic.

By combining these principles, the framework provides not only a powerful simulation engine but also a practical and accessible interface for modeling and evaluating the dynamics of hybrid human-machine systems in industrial settings.

1.2 Justification of the Selected Toolchain

To effectively simulate complex interactions between human operators and automated systems within industrial assembly environments, it was necessary to select a set of development tools that offered full control, adaptability, and transparency. For this reason, the framework was developed entirely in Python using two key open-source libraries: SimPy for discrete-event simulation and Qt (via PyQt5) for building the graphical user interface.

1.2.1 Open-Source Motivation and Control over Logic

The primary motivation for selecting Simpy and Python as the simulation backbone was to retain full control over the modeling logic and system behavior during the development process. By avoiding proprietary or closed-source platforms, it became possible to design a fully transparent simulation framework, where all elements, such as machines, operators, buffers, and error and statistical conditions, can be defined with full flexibility.

SimPy's process-oriented paradigm and event-driven core are particularly well-suited for modeling discrete operations, such as processing times, operator availability, buffer delays, and routing between machines. While some initial learning was necessary, SimPy ultimately proves to be a robust foundation for building a custom simulation engine tailored to the specific needs of human-machine collaboration.

Using an open-source foundation eliminates licensing concerns, enabling the framework to be used, adapted, and scaled freely within the company. SimPy's permissive MIT license further supports this advantage by permitting unrestricted use, modification, and distribution, even in commercial applications, without imposing legal or proprietary limitations. In an industrial context, all developed logic belongs to the organization that adopts the tool. Additionally, open-source development enhances long-term maintainability, as there are no external dependencies that could hinder the framework's evolution.

1.2.2 Graphical Interface Suitability

To complement the simulation logic and improve usability, a custom graphical user interface (GUI) was developed using Qt framework (PyQt5). This was done to provide an intuitive, visual environment in which users, particularly industrial engineers, could model assembly lines without writing code. PyQt5's 2D graphical and visual elements enable users to drag, position, and connect elements. The flexibility of Python and Qt meant that the interface could be designed specifically for this use case, free from the limitations typically imposed by general-purpose simulation platforms.

1.2.3 Limitations of Existing Tools

Several commercial simulation platforms were initially considered during the early stages of the project, including FlexSim and AnyLogic. While these tools provide powerful features and graphical environments, they were ultimately deemed unsuitable for this application for several reasons. Firstly, the licensing models of these platforms restrict the ability to freely distribute or modify the simulation logic. In some cases, even exporting a fully defined model required a paid version of the software. Secondly, many of the specific elements required for modeling hybrid human-robot assembly lines, such as manual task phases, semi-automatic logic, and operator allocation rules, were either missing or available only through paid extensions.

Additionally, these platforms often lack transparency in how simulation entities behave under the hood, making it difficult to adjust or validate model assumptions. In contrast, developing a framework from scratch using Python allowed for full transparency, unlimited extensibility, and zero licensing cost.

1.3 Architecture Overview and Module Responsibilities

The simulation framework was designed in a modular architecture, ensuring a clear separation of responsibilities between the graphical user interface, simulation logic, and data handling. Each part of the framework is organized into separate files, with functionality encapsulated in distinct classes. This structure ensures clear separation of concerns, making the project easier to maintain, reuse, and extend in the future.

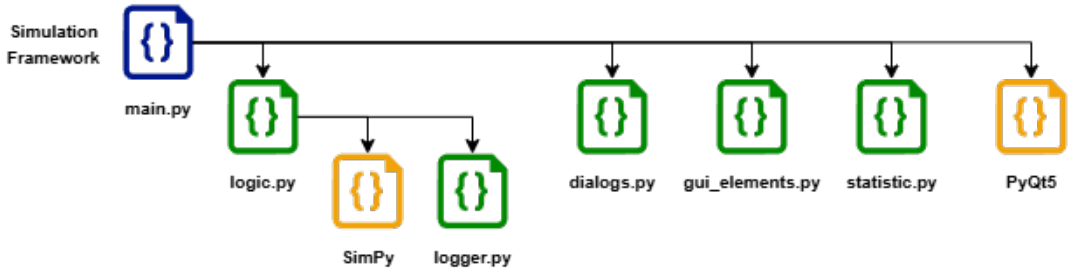


Figure 1.1: Simulation Framework Python File Hierarchy

At the core of the application lies the **main.py** module, which initializes and orchestrates the user interface, built upon the PyQt5 framework. It is responsible for creating the main application window, rendering the visual workspace, and managing user interactions such as adding elements, saving/loading configurations, launching simulations, and handling tab management for multiple simulation scenarios.

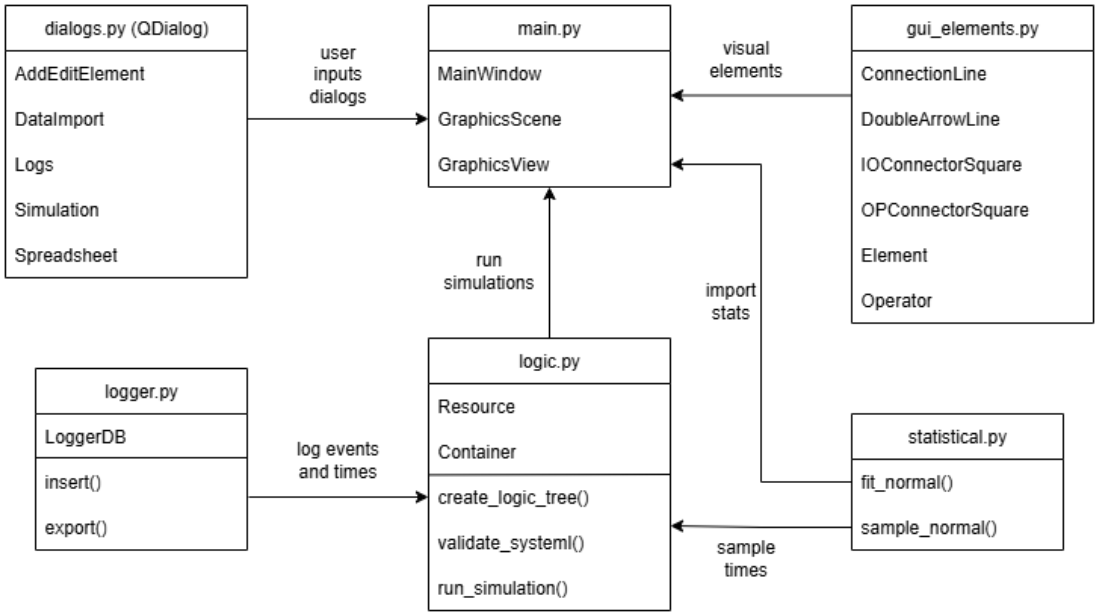


Figure 1.2: UML 'Architecture' Diagram (Shortened Version)

To support a visually intuitive experience, **main.py** interacts closely with two key modules: **gui_elements.py** and **dialogs.py**. The first defines the visual components of the assembly line, with graphical representations of logic nodes and connectors, all implemented through classes that inherit from QDialog. These elements represent various simulation entities such as machines, buffers, conveyors, operators, inputs/outputs, and connectors, that can be placed and configured on a 2D canvas. The **dialogs.py** module handles all dynamic dialog windows, including those for

adding or editing elements, setting simulation parameters, and displaying success or error messages.

The **logic.py** module translates the graphical elements into SimPy entities and manages the event-driven simulation environment. It executes the process flow of each piece, handles resources, assigns operators to tasks, and controls the timing of operations based on user-defined parameters. Additionally, it includes a system validation function to ensure connectivity and consistency before running the simulation.

The **statistics_func.py** module acts as a utility layer for probabilistic modeling, providing functions to fit normal distributions from data and to sample process and loading times during the simulation for processes.

1.4 Discrete-Event Simulation with SimPy

The simulation core of the framework is built using SimPy, a process-based discrete-event simulation library for Python. SimPy allows the modeling of asynchronous, event-driven operations, where multiple resources interact dynamically. This section details the main simulation entities, the model validation strategy, and the modeling logic used to simulate the behavior of hybrid assembly lines.

1.4.1 Simulation Entities

In logic.py, each modeled system is represented through a set of modular logical entities, each corresponding to a key physical component of an industrial assembly line. These entities are implemented as classes in the module, allowing for easy reuse and parameter customization. They utilize SimPy primitives like Resource and Container to model behavior and interactions.

A **simpy.Resource** models limited capacity resources that must be requested before use and released afterward. A **simpy.Container** represents a fixed-capacity storage unit, making it ideal for modeling buffers or inventories.

- **Operator** *(simpy.Resource)* - Models a human resource responsible for performing manual or assisted tasks in processes and transport operations.
- **Source** *(simpy.Resource)* - Represents the entry point of the system, where raw materials are introduced. Each simulation instance starts with the creation of piece processes in this Element.
- **Process** *(simpy.Resource)* - Represents a transformation operation within the assembly line flow. A process can be configured as automatic, manual, or semi-automatic, enabling the simulation of tasks performed by machines, human operators, or cobots.

- **Buffer** *(simpy.Container)* - Stores intermediate parts temporarily between processing stages. Buffers are used to decouple timings and manage flow accumulation under variable processing durations.
- **Conveyor** *(simpy.Container)* - Represents a real conveyor responsible for transporting materials between processes. It can also be seen as a moving buffer that requires operator intervention for loading, while the unloading phase depends on the type of the subsequent Element in the process.
- **Target** *(simpy.Resource)* - Represents the endpoint of the assembly line. When a piece reaches a Target, it is marked as completed, and its process is terminated from the simulation.

1.4.2 Model Validation and Simulation Parameters

Before execution, the framework performs a structural node flow validation of the assembly line model. This step ensures the integrity and feasibility of the scenario for simulation:

- It verifies that at least one **Source** and one **Target** exist.
- It checks that all elements are correctly connected.
- It confirms that all assigned operators are valid and logically placed.
- It ensures that every possible path from a **Source** leads to at least one **Target**, forming a closed, executable system.

This validation is implemented in the ***validate_system()*** function in **logic.py** and is automatically triggered before the simulation starts. Users are informed of issues through the graphical interface.

Beforehand, the user-created visual assembly line is internally converted into structured logical nodes by the dedicated function ***create_logic_tree()***, which scans the canvas, classifies each Element, and builds the graph-based data model with properties like process type, assigned operators, time distributions, buffer capacities, and I/O connections.

1.4.3 Process-Based Modeling Concepts

The simulation follows a process-oriented modeling approach, where each piece acts as a discrete process moving through the assembly system. This allows for concurrency and flexibility, with each entity independently handling its flow and interactions with resources.

In SimPy, the simulation environment is created using an instance of **simpy.Environment**, which manages time progression and serves as the central event scheduler. It executes all processes and events in a **FIFO** order based on their scheduled times, ensuring chronological accuracy and resolving resource contention based on arrival times.

The core building blocks of SimPy include:

- **env.process()** — starts a new concurrent process in the environment
- **yield env.timeout(t)** — suspends a process for **t** units of simulated time
- **resource.request()** — requests exclusive access to a resource
- **yield resource.request()** — blocks the process until the resource is available
- **resource.release()** — returns the resource to the pool
- **env.run(until=x)** — runs the simulation until a given time or event

This modeling approach enables entities like operators and machines to operate independently while competing for shared resources within the system. Each entity follows its own logic and timing, creating a highly interactive and realistic simulation environment. As these entities progress through the system, resources are dynamically allocated and released based on availability and demand. This configuration enables complex behaviors to naturally emerge, such as delays from resource contention, bottlenecks due to process queues, and congestion resulting from overlapping tasks.

1.4.4 Simulation Flow and Resource Scheduling

In the context of this framework, each piece begins its life cycle at a Source, where a corresponding SimPy process is instantiated to process its route through the assembly line. This process encapsulates the sequence of operations, resource requests, delays, and transitions that the piece will undergo until it reaches a **Target**.

At each step in the flow, the piece performs the simplified sequence of operations:

1. **Request access** to the current node in the path.
2. **Request an operator**, if the current process requires human intervention.
3. **Simulate a delay** corresponding to process time or handling time.
4. **Release a resource** once the operation is complete.
5. **Move to the next node**, following the defined path.
6. **Simulate transport**, transitioning to the next process.
7. **Release or retain an operator** based on the defined flow strategy.

The movement is dynamically defined at runtime, with each piece flowing independently and automatically queuing if resources are unavailable. Operators are created has discrete resources that can serve multiple elements depending on the scenario. The algorithm is equipped with safeguards to prevent over-allocation, and SimPy provides fair scheduling through its built-in queue mechanism. This process continues until a piece reaches the **Target**, at which point it is logged and its process is terminated. The simulation concludes either when a predefined number of pieces have been processed or when the specified simulation time has elapsed.

1.4.5 Time Handling

All time-related behaviors in the simulation are driven by stochastic models using normal distributions, defined for each Element and configured through the graphical interface or imported from real empirical datasets provided by Bosch for the case study.

The following durations are simulated:

- **Processing time**: Task duration per operation manual or automatic.
- **Loading/Unloading time**: Represents times to remove or insert pieces in machinery.
- **Transport time**: Movement times of parts as they travel through the assembly line.

Distributions are defined by a mean (**mu**) and standard deviation (**sigma**) and sampled using functions from ***statistics_func.py***. These simulated times help enhance realism by replicating the variability seen in real-world operations.

1.4.6 Error Events and Failure Simulation

To more accurately reflect the dynamic and uncertain nature of real-world assembly systems, the simulation framework incorporates two complementary event types related to operational reliability: piece conformity errors and process failures due to breakdowns.

Each process in the system includes an optional error rate parameter, which defines the probability that a processed part will be rejected due to non-conformity. This rate can be configured independently for each node through the graphical interface and represents issues such as operator-detected defects or rejections from quality tests. During simulation, each time a part completes its operation at a node, a random draw based on the defined error rate percentage determines if the piece is classified as defective.

If a defect is detected, the piece is immediately removed from the system and registered in the logs with a **Rejected** comment. It does not continue through the remaining nodes, and no further resources are allocated to it. This behavior reflects the logic commonly applied to automated quality control stations, particularly test machines, where faulty units are discarded on the spot.

The framework also supports the simulation of unexpected machine failures, representing unplanned downtimes such as mechanical failures or maintenance interruptions. This behavior is controlled by the following parameters:

- **MTBF** - average time between machine failures, relative to manual or semi-automatic processes or conveyors.
- **MTTR** - average time to repair.
- **MTBM** - average time between machine maintenance, relative to manual or semi-automatic processes or conveyors.
- **TSLM** - time since the last maintenance or repair was completed.
- **MTM** - average time required for maintenance completion.

These values can be configured by the user in the process configuration dialog. Internally, the simulation engine handles unexpected failures and scheduled maintenance as parallel mechanisms. Failures are triggered probabilistically based on the **MTBF** parameter, and result in the node being temporarily blocked for the duration defined by the **MTTR**. During this time, no new part may enter the process, and any pending tasks must wait.

When the **TSLM** value reaches the **MTBM** threshold, a preventive maintenance is triggered. This also blocks the node for the specified **MTM** duration. Importantly, the completion of a maintenance event resets the failure timer, postponing the occurrence of subsequent breakdowns. This model implements realistic Bosch OVar industrial practices, incorporating preventive maintenance to reduce the risk of unplanned failures.

Both types of downtime are automatically managed by the simulation engine, with all related events recorded in the log files. This enables users to evaluate how equipment reliability strategies affect overall system performance.

1.5 Graphical User Interface with Qt

The simulation tool includes a graphical user interface built with Qt that allows users to create, configure, and manage simulation scenarios visually. Instead of writing code, the user can drag and connect elements directly on a 2D canvas, assign operators, adjust simulation parameters, and validate the system before execution. The interface is organized to provide quick access to all key functionalities, such as element management, file handling, and simulation control, making it easier to build and run models that reflect real assembly line systems.

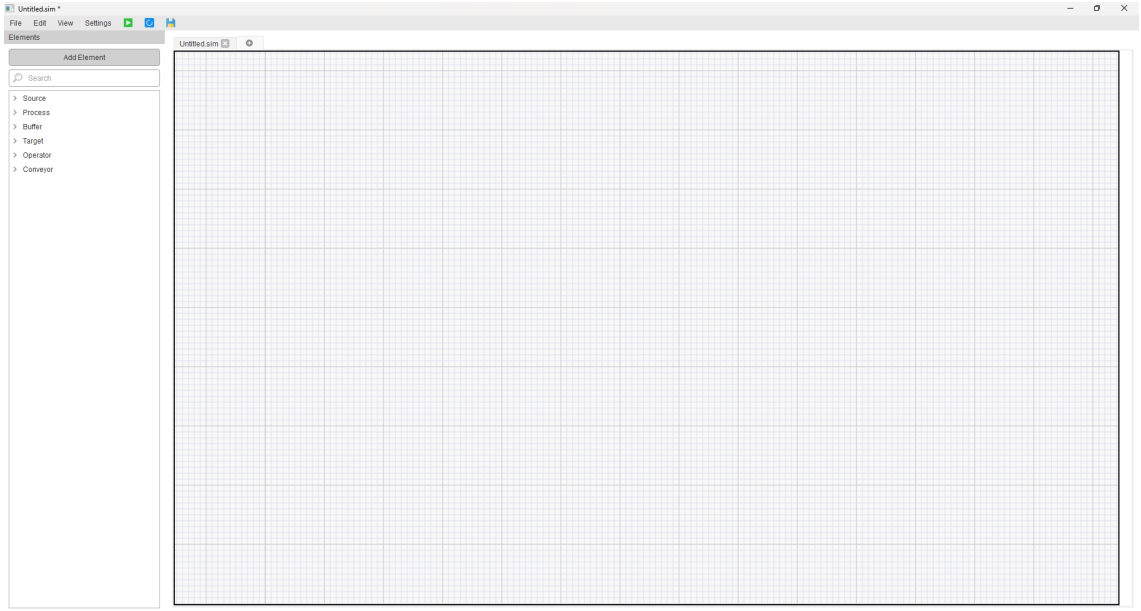


Figure 1.3: Simulation Framework Full Window

1.5.1 MainWindow Layout and Controls

The main window is the central component of the interface, integrating all primary actions for simulation creation, editing, validation, and execution. Its layout follows a familiar desktop application structure, combining a menu bar, toolbar icons, and a centered canvas for scenario overview.

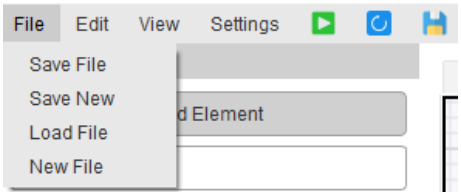


Figure 1.4: Top Menu Bar

At the top of the interface, a menu bar offers access to textual options. Currently, only the File menu is implemented, providing basic file operations:

- **Save File**: Saves the current scenario, either by overwriting or by creating a new filename if none exists.
- **Save New**: Saves the current scenario in complete new file.
- **Load File**: Loads an existing .sim file and reconstructs the corresponding scenario.
- **New File**: Opens a new blank scenario in a separate tab.

Next to the menu bar, three icon buttons allow quick access to core simulation operations:

- The **green play button** opens the simulation parameters dialog and allows the simulation to be run. Internally, it first validates the current model and, if successful, starts the simulation engine using the parameters defined by the user.
- The **blue validation button** checks the logical structure of the modeled system without executing the simulation, providing immediate feedback in case of missing connections or invalid configurations.
- The **save button** simulates the operation of the previously mentioned "Save File" button.

1.5.2 Element Palette and Canvas

On the left side of the interface, the element palette is organized into a vertical panel that allows users to add, search, and browse the elements required to model an assembly line.

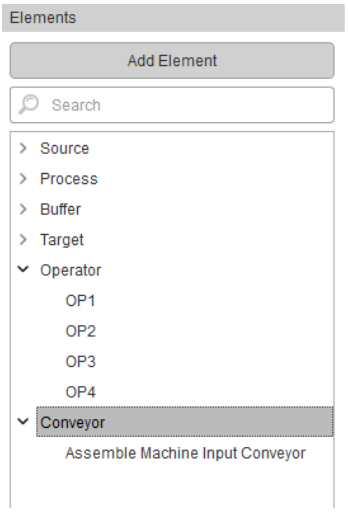


Figure 1.5: Elements Tree

At the top of the panel, the **AddElementt** button opens a configuration dialog where the user can define the properties of a new element.

The search bar allows filtering the element tree by either the name or class type, facilitating quick access in large projects. The element tree itself is structured by class, grouping elements together for easy navigation. The user can expand or collapse each class to view the available elements within.

To place an element on the workspace, the user can simply double-click it in the list, which spawns a copy directly onto the canvas.

The central area of the interface is the canvas, implemented using Qt's **QGraphicsScene** and **QGraphicsView**. This is where the simulation model is visually constructed, better viewed in Figure 1.4. Users can:

- Drag elements to reposition them freely.
- Connect elements using interactive I/O squares and operator connectors.
- Zoom in and zoom out, and move through the canvas.
- Right-click on any element to access a context menu with options to delete or edit its properties.

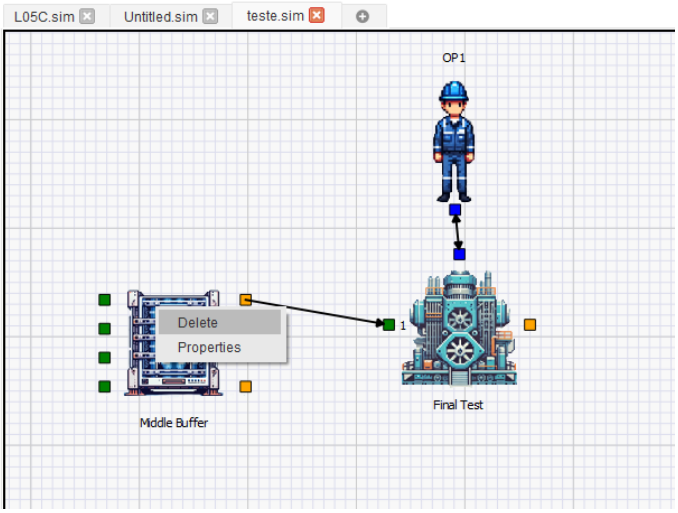


Figure 1.6: Canvas With Elements

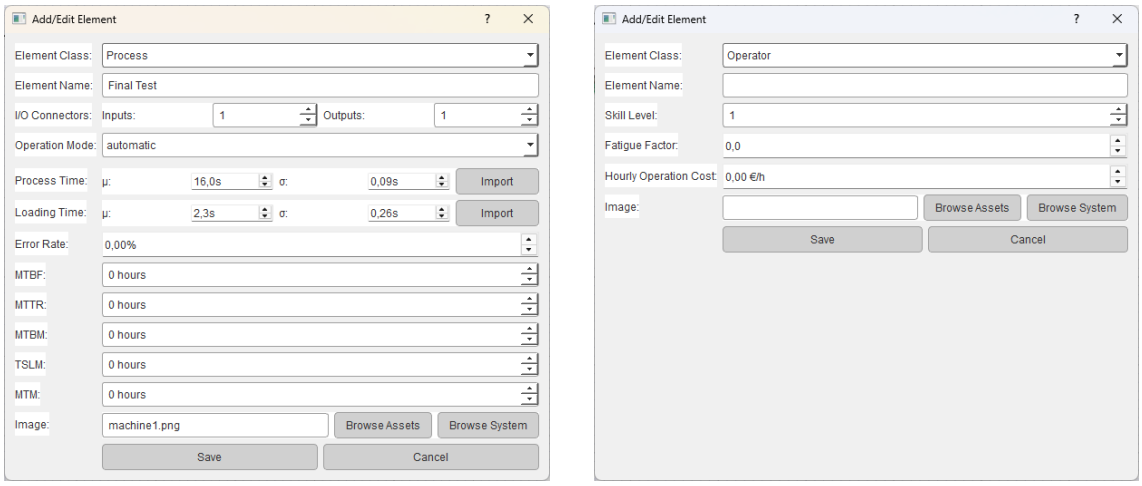
Above the canvas, a tab bar enables (just like a web browser) users to manage multiple scenarios within the same session. Each tab represents an independent scene, allowing separate configurations to be developed and tested in parallel.

1.5.3 Dialogs and Parameters

To configure the behavior of each element and define key simulation parameters, the interface makes use of a set of dialog windows that are displayed contextually as the user interacts with the model. These dialogs simplify data entry and enforce structure, ensuring that all necessary parameters are defined before execution.

Add/Edit Element Dialog

Whenever a user creates a new element using the **Add Element** button or edits an existing one from the canvas, the application opens the **Add/Edit Element Dialog**. This dialog automatically adjusts its layout based on the class of the selected element. The interface is organized into sections, with each section representing a specific group of parameters.



(a) AddEditElement Dialog Window - Process (b) AddEditElement Dialog Window - Operator

Figure 1.7: AddEditElement Dialog Windows

Time-based settings use paired input fields for average and standard deviation. When necessary, users can import time values directly from Excel or other sources by pasting numeric data into the input fields. The application then automatically fits a normal distribution to the imported values using statistical tools from **statistics_func.py**.

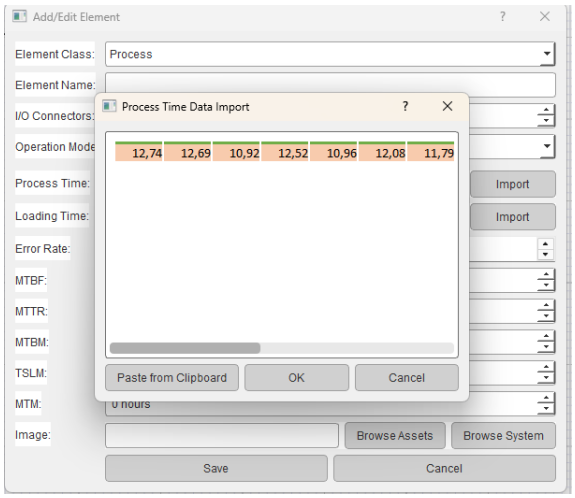
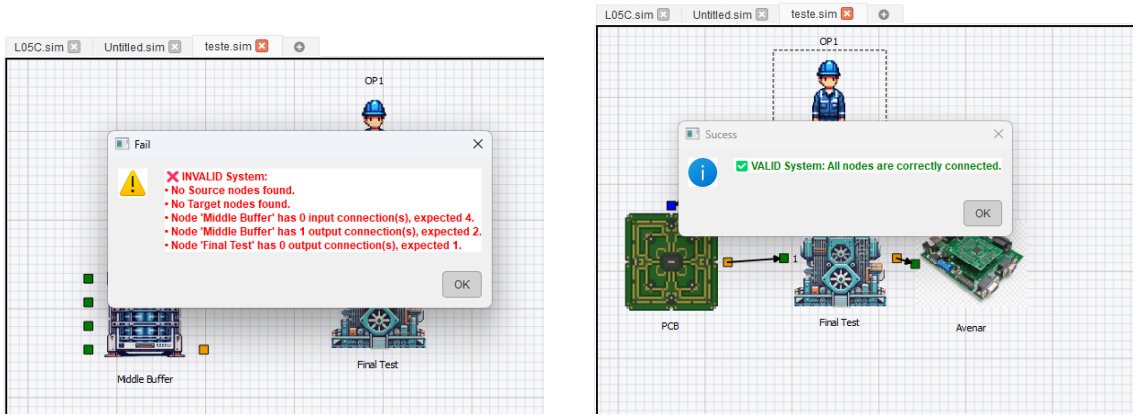


Figure 1.8: Process Time Import Dialog

Validation Feedback

When the blue validation button is pressed, the application performs a series of logical checks on the current model, described in Section 1.4.2.



(a) Validation Window - Error

(b) Validation Window - Success

Figure 1.9: Validation Window

Feedback is provided in a pop-up message. If the model is valid, a success message appears. If issues are found, the dialog lists all errors and guides the user to correct them. This validation logic is also invoked automatically when the user attempts to run a simulation

Simulation Parameters Dialog

Before running the simulation, a dedicated dialog prompts the user to define key global parameters that control the simulation runtime and evaluation:

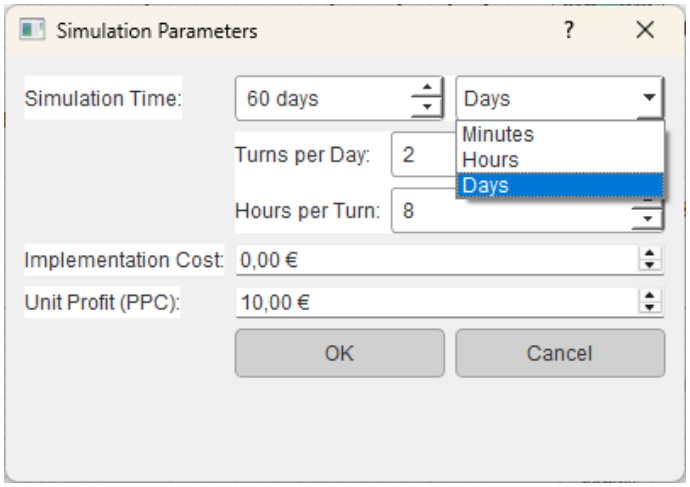


Figure 1.10: Simulation Parameters Dialog

- **Simulation Time**: The user selects whether the simulation is defined in minutes, hours, or days.

  If **Days** is selected:

  - **Turns per Day** and **Hours per Turn** are shown to determine the working time per day.

- **Implementation Cost**: A fixed value used later for cost-benefit analysis.
- **Unit Profit** (referred to as **PPC** at Bosch): The gain per successfully completed part.

These parameters are passed to the simulation engine as arguments.

1.6 Output Metrics and Logging Mechanism

To support post-simulation analysis and scenario comparison, the logic component of the framework records detailed data throughout the execution of each piece's process. All relevant events, including part movements, process durations, and operator interactions, are automatically logged and exported in structured formats. These logs form the foundation for calculating KPIs and enable engineers to thoroughly analyze the modeled system behavior. The following sections explain how this data is organized and which metrics can be extracted from the recorded results.

1.6.1 Logging Architecture and Export Formats

The simulation framework includes an internal logging system designed to register all relevant events that occur during the execution of a scenario. Two distinct logs are maintained in parallel: one for the behavior of each piece as it flows through the assembly line, and another for the operators, tracking their activity and allocation over time. These logs are automatically extracted and saved once the simulation concludes and are presented to the user through popup spreadsheet interfaces, which can exported in the Excel format (**.xlsx**).

Part Flow Logging

Each time a piece enters, moves through, or exits a node in the assembly line, a log entry is created in the part log.

- The **ID** of the piece (unique identifier),
- The **node** name involved in the event,
- The current total **cycle time** this piece has spent in the system.,
- The cumulative simulation **process time**,

- A **comment** describing the action.

	ID ↑	NODE	CYCLE_TIME	PROCESS_TIME	COMMENT
1	1.1.1.1	PCB	0.0	0.0	In
2	1.1.1.1	PCB	0.0	0.0	Transport to Stick a Label
3	1.1.1.1	Stick a Label	0.0	0.0	Input Inserted Need:1 Have:1
4	1.1.1.1	PCB	0.0	0.0	Out
5	1.1.1.1	Stick a Label	0.0	0.0	In
6	1.1.1.1	Stick a Label	0.0	0.0	Manual Work Start
7	1.1.1.1	Stick a Label	3.3	3.3	Manual Work End
8	1.1.1.1	Stick a Label	3.3	3.3	Transport to Assemble Machine Input Conveyor
9	1.1.1.1	Stick a Label	3.3	3.3	Out
10	1.1.1.1	Assemble Machine Input Conveyor	3.3	3.3	In
11	1.1.1.1	Assemble Machine Input Conveyor	3.3	3.3	Conveyor Transport
12	1.1.1.1	Assemble Machine Input Conveyor	3.3	3.3	Transport to Assemble Components
13	1.1.1.1	Assemble Components	3.7	3.7	Input Inserted Need:1 Have:1
14	1.1.1.1	Assemble Machine Input Conveyor	3.7	3.7	Out

Figure 1.11: Spreadsheet Piece Log

This log allows reconstructing the complete trajectory of each piece across the system and analyzing bottlenecks, machine and station times, or excessive cycle times. The data is sorted by process time but can be reordered or filtered interactively in the output popup or through external spreadsheet tools.

Operator Activity Logging

In parallel, the framework logs all interactions involving human resources. For each operator, the log records:

- The operator **ID** and **name**,
- The **node** where the **interaction** occurred,
- The type of **action**,
- The simulation **process time** at which the action took place,
- The **piece ID** of the associated.

	ID	NAME	NODE	TYPE	PROCESS_TIME ↑	PIECE
1	26.0	OP1	PCB	Request	0.0	1.1.1.1
2	26.0	OP1	Stick a Label	Release	0.0	1.1.1.1
3	26.0	OP1	Stick a Label	Request	0.0	1.1.1.1
4	26.0	OP1	Stick a Label	Hold for Conveyor	3.3	1.1.1.1
5	26.0	OP1	Stick a Label	Release	3.3	1.1.1.1
6	26.0	OP1	Assemble Machine Input Conveyor	Request	3.3	1.1.1.1
7	26.0	OP1	Assemble Components	Release	3.7	1.1.1.1
8	26.0	OP1	PCB	Request	3.7	1.1.2.1
9	26.0	OP1	Stick a Label	Release	3.7	1.1.2.1
10	26.0	OP1	Stick a Label	Request	3.7	1.1.2.1
11	26.0	OP1	Stick a Label	Hold for Conveyor	7.5	1.1.2.1
12	26.0	OP1	Stick a Label	Release	7.5	1.1.2.1
13	26.0	OP1	PCB	Request	7.5	1.1.3.1
14	26.0	OP1	Stick a Label	Release	7.5	1.1.3.1

Figure 1.12: Spreadsheet Operator Log

This structure enables evaluation of operator utilization, identification of processes that rely heavily on manual labor, and tracking of how long operators remain active or idle during the simulation. These logs are especially valuable for pinpointing stations that may be overburdened or underutilized in terms of human resources.

Export and Visualization

Both logs are exported automatically when the simulation ends. The logs are presented to the user in dedicated spreadsheet pop-up windows, where they can be reviewed immediately without the need to access the raw files. The export is done in Excel format (.xlsx) to preserve formatting.

For improved interoperability, the logs can be post-processed to reorder entries and format the data by aligning events chronologically, and grouping by piece ID or operator. These logs form the primary source for extracting quantitative indicators, as detailed in the following section.

1.6.2 Key Performance Indicators

The structured logs generated by the simulation framework provide the foundation for extracting a variety of performance metrics. These indicators are essential for assessing the efficiency, workload balance, and potential constraints of a given assembly configuration. Some KPIs are calculated automatically at the end of the simulation and shown to the user, while others are derived from interpreting the exported data.

For the **Automatically Extracted Metrics**, the framework calculates several KPIs by default based on the events registered during execution, such as:

- **Process Task Time**
  The time each part spends in individual processes is recorded and aggregated, allowing for an immediate assessment of process-level efficiency.
- **Station Cycle Time**
  A "station" is defined as a set of processes sharing the same operator. The total time a piece spends across all processes within a station is calculated, providing insight into how this station affects throughput.
- **Hold Time**
  Time spent by a piece waiting in buffers or being delayed due to unavailable resources is tracked explicitly. This allows engineers to quantify non-value-adding time in the system.
- **Rejection Rates**
  The simulation framework tracks rejected parts and calculates rejection ratios globally, per station, and per process. These values are derived from the system logic and help identify areas prone to failure or instability.
- **Implementation Cost and Economic Return**
  During simulation setup, users can define two key economic parameters: the implementation cost of the proposed system configuration and the profit per completed piece. These values are saved with the simulation parameters and used to estimate the economic viability of each scenario.

While the logs provide all the necessary data, certain **Engineer-Driven Metrics** require manual interpretation or external post-processing by engineers to extract deeper insights from the recorded simulation results.

- **Bottleneck Identification**
  By analyzing **Station Cycle Times**, operator engagement, **Hold Times**, and **Process Task Times**, engineers can identify bottlenecks within specific processes or stations. These metrics identify areas where the system operates inefficiently, enabling engineers to detect underperforming points and resource imbalances.
- **Operator Utilization**
  The operator log can be used to estimate active versus idle time per resource, enabling evaluations of workload balance and potential underuse or overworkload.

- **Throughput and Flow Balance**
  By comparing entry and exit times across parts and nodes, engineers can calculate average system throughput and detect imbalances, such as areas where buffers become overloaded with pieces, indicating inefficiencies in the configuration.

1.7 Model Assumptions and Known Limitations

While the simulation framework provides a detailed and functional representation of assembly systems, it necessarily incorporates a number of assumptions and simplifications that influence its behavior and scope. These choices were primarily driven by the data provided by Bosch, practical constraints observed during line visits, and the objective of creating a decision-support tool that is both flexible and maintainable.

1. **Behavior of Operators**
   Human decision-making is inherently autonomous and reactive, often influenced by experience, intuition, and contextual awareness. However, the current simulation models operator behavior in a deterministic and linear fashion: each operator proceeds with tasks in sequence, moving from one assigned element to the next, unless blocked by an automatic machine or by an unavailable downstream node. When blocked, the operator selects the oldest pending task among the accessible ones. This approach is consistent with practices observed on the shop floor at Bosch, where task execution is mostly sequential and driven by proximity and availability rather than dynamic prioritization.
   In terms of operator profiles, no distinctions were made regarding experience or skill levels, for example: the operator "OP1" represents the average performance of multiple workers rotating through the assigned station. Consequently, the simulation does not account for variations in task speed, error rates, or decision-making among individual human resources.

2. **Simulation of Collaborative Robots**
   While the tool allows modeling semi-automatic processes—where both machine and operator are involved—these behaviors were implemented based on field observations and internal assumptions. Due to the lack of controlled experiments and specific performance data for cobots in the studied lines, their behavior was only theorized, not validated.

3. **Physical World Modeling**
   The simulation provides a reasonably accurate temporal representation of the real assembly system, especially in terms of process duration, operator use, and flow constraints. However, it does not capture all aspects of physical layout, safety procedures, or ergonomic factors.

4. **Artificial Intelligence and Digital Twin System**
   From a technical standpoint, the framework does not include automatic balancing, intelligent routing, or optimization features. All decisions related to layout, operator assignment, and system configuration are left entirely to the user. This emphasizes the tool's role as a decision-support system rather than a digital twin. Although it draws on digital twin concepts like scenario exploration and virtual testing, the framework lacks real-time synchronization, bidirectional feedback, and autonomous control, which are essential characteristics of a true digital twin.